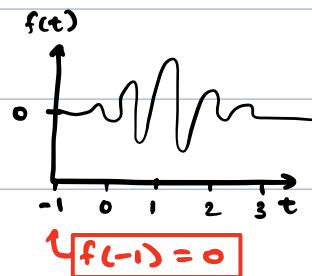


Signal Processing

signal $\longrightarrow$ function of time!

$f(\cdot)$ $\longrightarrow$ whole signal function

$f(t)$ $\longrightarrow$ value of signal at time t
VALUE of a function!



domain: set of points input t can be

e.g. uniformly sampled at X KHz

discrete-time: $t = 0, \pm 1, \pm 2, \dots$

$\uparrow$ "epochs" not necessarily seconds

$f(t)$ $\left\{ \begin{array}{l} \text{value} \\ \text{vector-valued} \\ \text{boolean} \end{array} \right.$

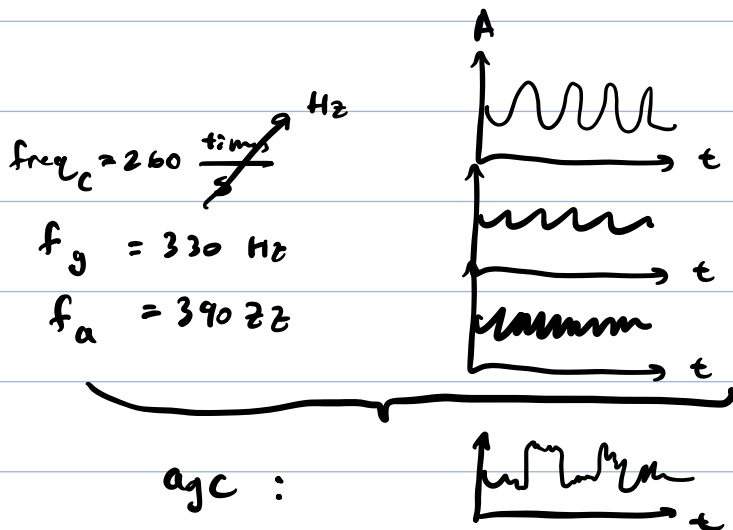
$$\vec{V}(t) = \begin{bmatrix} v_1(t) \\ v_2(t) \\ v_3(t) \end{bmatrix}$$

$\uparrow$
 vector
 valued
 signal
 function

$\swarrow$
 scalar
 values

Fourier Transform

signal fn.
in time $\longrightarrow$ FT $\longrightarrow$ signal fn.
in frequency



$\longleftarrow$ Time

Convolutions

$$(x * g)[k] = \sum_j x_{k-j} g_j$$

$$x = [x_0, x_1, x_2, x_3, x_4]$$

input sequence

convolving filter

$$g = [g_2, g_1, g_0]$$

length of convolution

$$(x * g)[k] = \sum_{j=0}^2 x_{k-j} \cdot g_j$$

$$(x * g)[0] = x_0 g_0 + \cancel{x_1 g_1} + \cancel{x_2 g_2} = x_0 g_0$$

$$(x * g)[1] = x_1 g_0 + x_0 g_1 + \cancel{x_1 g_2} = x_1 g_0 + x_0 g_1$$

$$(x * g)[2] = x_2 g_0 + x_1 g_1 + x_0 g_2 = x_2 g_0 + x_1 g_1 + x_0 g_2$$

Notice:

- if I treat inputs as polynomial coefficients, the computation is identical!

$$f(z) = x_0 + x_1 z + \dots + x_n z^n$$

$$g(z) = g_0 + g_1 z + g_2 z^2$$

$$h(z) = f(z)g(z) = (g_0 + g_1 z + g_2 z^2)(x_0 + x_1 z + \dots + x_n z^n)$$

$$= h_0 z + h_1 z^2 + h_2 z^3 + \dots$$

coefficient at index i

is $(x * g)[i]$!



Convolution == polynomial multiplication

Our goal is to compute a convolution FAST.

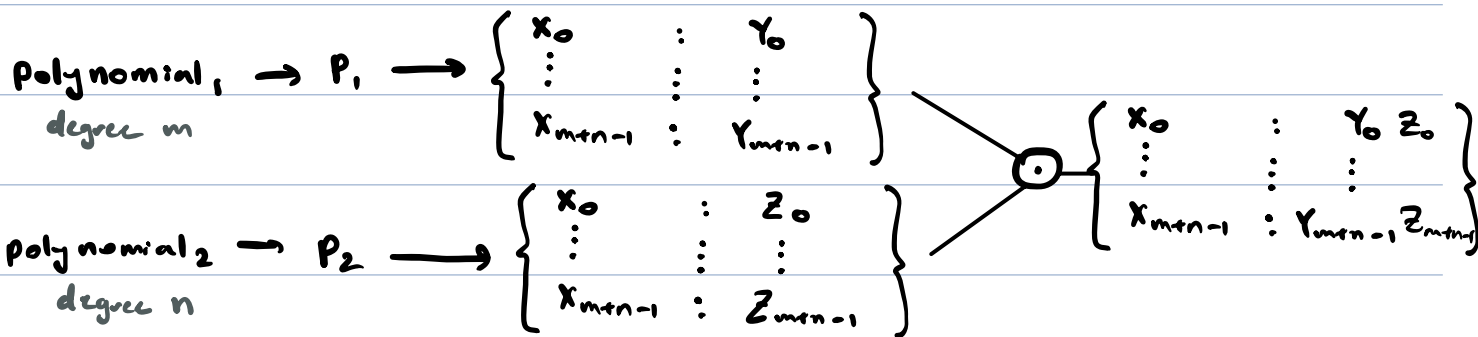
① Sliding window: Slow!

② Expanding out a polynomial is $O(n^2)$

we need a faster way of multiplying 2 polynomials!

Algorithm:

- important fact: given polynomial of degree n , it can be uniquely defined by $n+1$ points.



Eval $n+m$ points
for P_1 & P_2
(take FFT)

pointwise multiply
we now have $n+m$
points evaluated for
polynomial $P_1 P_2$!

we have a polynomial in value-form:
we need them in coefficient form!

Soln: take inverse FFT!

Polynomial 3

1) FFT the seq & filter
value-form

2) Pointwise multiply

3) Take iFFT
coefficient-form

These coefficients are the
convolution results we want!

The tricky part is doing the Coeff. $\rightarrow$ Value form transformation:

Naively (coefficient $\rightarrow$ value form):

Vandermonde matrix $\left\{ \begin{bmatrix} a_0 & \dots & a_0^n \\ a_1 & \dots & a_1^n \\ \vdots & & \vdots \\ a_n & \dots & a_n^n \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} f(a_0) \\ f(a_1) \\ \vdots \\ f(a_n) \end{bmatrix} \right.$

$\uparrow$ $\uparrow$ $\uparrow$

n data points Polynomial Coefficients n data points [Output]

[input]

Takes $O(n^2)$ to compute these outputs

(value $\rightarrow$ coefficient form):

solve $\vec{x} = A^{-1} \vec{f(a_i)}$

Takes

- $O(n^2)$ to invert this special matrix
- $O(n^2)$ to multiply w/ this vector

TRICK: by selecting special points,
the FFT & iFFT lets you
convert btwn these two forms
in $O(n \log n)$ time!

$\rightarrow$ $\star$ Perform long convs. fast!